2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS) | 978-1-6654-1494-4/21/$31.00 ©2021 IEEE | DOI: 10.1109/IWQOS52092.2021.9521336

pSAV: A Practical and Decentralized Inter-AS Source Address Validation Service Framework

Jiamin Cao*†, Ying Liu*†, Mingxing Liu*†, Lin He*†, Yihao Jia‡, Fei Yang‡
*Institute for Network Sciences and Cyberspace, Tsinghua University
†Beijing National Research Center for Information Science and Technology (BNRist)
‡Huawei Technologies

***Abstract*—Source IP address spoofing has been a major vulnerability of the Internet for many years. Although much work has been done to study the problem extensively, spoofing continues to occur frequently and has led to many serious network attacks. Inter-AS source address validation (SAV) is considered an important defense method for AS to filter spoofed packets. However, existing work has been unable to drive inter-AS SAV deployment into practice due to the lack of deployment incentives and trust foundation.**

In this paper, we propose a practical and decentralized inter-AS SAV service framework, *pSAV*, to promote inter-AS SAV deployment. *pSAV* increases deployment incentives by treating SAV as a payable service and dividing the participant ASes into service subscribers, providers, and auditors. On the control plane, *pSAV* leverages blockchain as a trust foundation to provide service subscriptions and audits with automatic incentive allocation. On the data plane, *pSAV* leverages P4-programmable switches to provide flexible and high-performance SAV services. We prototype the *pSAV* control plane based on Hyperledger Fabric and implement various SAV techniques on Barefoot Tofino switches. The evaluation results show that (1) on the control plane, *pSAV* blockchain can provide high-performance service transactions (hundreds of transactions per second with second latency), and (2) on the data plane, *pSAV* can provide various high-throughput (hundreds of Gbps) SAV services using only one programmable switch.

***Index Terms*—Source address validation, Blockchain, Decentralization, IP spoofing**

I. Introduction

Source IP address spoofing is a severe security concern that has plagued the current Internet for years [1]–[3]. By forging IP source addresses, attackers can hide their true identities while launching attacks. To solve the IP spoofing problem, [4] proposes Source Address Validation Architecture (SAVA), which divides the source address validation (SAV) system into three levels from bottom to top: access network, intra-autonomous system (AS), and inter-AS. Access network SAV technologies [5] prevent hosts from forging other hosts in the same subnet, while intra-AS SAV technologies [6], [7] constrain their users from sending spoofed packets to other ASes. Both levels of SAV can benefit the entire Internet, but except for the deployers themselves [8]. Instead, inter-AS SAV [9]–[13] is what protects deployer ASes from IP spoofing. By identifying and filtering spoofed packets sent to or flowing through themselves, they can defend against IP spoofing attacks from outside.

The highly fragmented ownership of network infrastructure makes the actual deployment of inter-AS SAV particularly difficult [14]. In general, inter-AS SAV deployment faces two main challenges. The first challenge is the lack of deployment incentives. Some SAV technologies are based on cooperation among various deployers. However, the effort (filtering packets) and the reward (benefits of deployment) may vary among ASes. Some deployers benefit less but pay more than others. For example, SPM [9] creates a security alliance. All ASes are expected to inspect packets from outside the alliance, while trusting packets from ASes inside the alliance. Therefore, ASes at the alliance boundary naturally have to bear more of the burden of inspecting packets. Then ASes at the boundary have less incentive to deploy SPM.

The second challenge is the lack of a trust foundation. Existing inter-AS SAV technologies usually guarantee the authenticity of control information under the premise of specification and trust. When the number of deployed nodes is small, the trust foundation among ASes is relatively reliable. However, as the number of deployed nodes increases, such a trust foundation is no longer reliable. For example, when a deployer AS maliciously declares the ownership of other ASes' IP addresses, legitimate packets sent by other ASes will be incorrectly filtered. Similarly, an AS never knows whether other ASes in the same alliance are complying with SAV regulations or breaking promises. All of the above hinders the widespread deployment of SAV technologies.

apf [15] proposes inter-AS SAV as a payable address (authenticity) protection service to solve the deployment challenge. A service subscriber AS requests services from a service provider AS by paying money, and the service provider is rewarded by filtering spoofed packets for the service subscriber. To some extent, apf increases deployment incentives by redistributing incentives, but it also creates two new problems. First, apf assumes that service providers will deliver SAV services as promised. However, in a real-world distributed inter-AS environment, providers are not necessarily trustworthy and the effectiveness of SAV services is difficult to verify. Second, service providers need to be flexible in providing various SAV services according to the real-time needs of subscribers, which means that the packet processing process has to be constantly changing. However, SAV technology can be very complex. Using software is flexible, but can bring considerable performance degradation, while hardware is high-performance,

This work was supported by the National Natural Science Foundation of China (Grant No. 61772307) and National Key Research and Development Program of China (Grant No. 2018YFB1800405). Lin He and Ying Liu are the corresponding authors.

978-0-7381-3207-5/21/$31.00 ©2021 IEEE

but requires frequent updates. That said, service providers may find it difficult to adapt to many different SAV technologies.

In this paper, we propose a practical and decentralized framework for inter-AS SAV service, *pSAV*. We draw on apf's concept of payable SAV services. On the control plane, we build an inter-AS SAV service subscription and auditing platform with automatic incentive allocation using blockchain as the trust foundation. First, *pSAV* introduces third-party ASes as service auditors to proactively verify the service quality of providers. Second, by leveraging smart contracts, *pSAV* ensures that auditors and providers are automatically rewarded when acknowledged by subscribers. Finally, since all transactions are recorded on the blockchain, *pSAV* holds providers accountable for their services. Also, *pSAV*'s blockchain provides an authoritative reference and guides other ASes to better subscribe to higher quality services. On the data plane, we leverage P4-programmable switches [16], [17] to flexibly support various SAV technologies. Compared to expensive dedicated hardware and low-performance software, programmable switches can provide high performance, including high throughput and low latency, at a low cost. The challenge is to implement complex SAV technologies on programmable switches, especially labeling-based SAV technologies, which rely on complex cryptographic algorithms. Naive reproduction of these algorithms on programmable switches takes up many hardware stages, which requires recirculation and reduces throughput. Instead, we use a combination of exact match tables to reduce hardware stages with a relatively large memory overhead.

In this paper, we make four three contributions.

- We propose *pSAV*, a practical inter-AS SAV service subscription and audit platform that drives SAV deployment.
- We present the *pSAV* design. On the control plane, *pSAV* leverages blockchain as a trust foundation to provide distributed service subscription and auditing. On the data plane, *pSAV* leverages P4-programmable switches to provide flexible and high-performance SAV services.
- We prototype *pSAV* control plane based on Hyperledger Fabric [18] and build an inter-AS test-bed with Tofino switches and servers. The evaluation shows that (1) *pSAV* control plane can support high-performance service transactions (hundreds of transactions per second with second latency). (2) *pSAV* data plane can support various SAV technologies with high throughput (hundreds of Gbps) with only one programmable switch.

II. *pSAV* Overview

pSAV aims to provide a practical inter-AS SAV service framework to increase deployment incentives. Figure 1 shows the architecture and workflow of *pSAV*.

Architecture. On the control plane, *pSAV* includes a blockchain-based platform that provides SAV service subscription and auditing among different ASes. Each AS can register as a *service subscriber*, *provider*, or *auditor* according to its own requirements and capabilities. Subscribers can subscribe to SAV services from any provider, *i.e.*, asking the provider

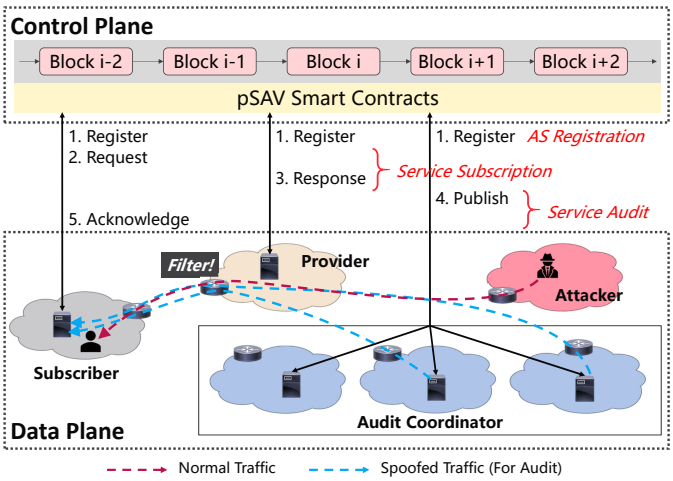


Fig. 1. **Architecture and workflow of *pSAV*.**

to deploy a specified SAV mechanism by paying money. All subscription transactions are public on the blockchain. Auditors verify the effectiveness of subscribed SAV services by actively spoofing IP addresses. After detecting a substandard SAV service, they publish the results on the blockchain. Once subscribers acknowledge their reports, they are paid, giving them an automatic incentive. Subscribers can then learn authoritative reviews of the effectiveness of various providers' services, which can further guide their future service subscriptions. Based on the service transactions on the control plane, providers issue specified SAV rules on their border routers on the data plane.

Workflow. (1) Each AS can be registered as a subscriber, provider, or auditor. In addition to the AS number (ASN) as the identity of each AS, subscribers shall provide their IP address prefixes, and providers shall provide their lists of available SAV services. (2) Subscribers obtain the service lists and choose to request services from the provider. (3) The corresponding provider may accept or refuse to provide the service. (4) Voluntary auditors obtain services from the blockchain, verify the effect of ongoing services, and submit the audit result to the blockchain. (5) If the subscriber acknowledges the validity of the audit results, the reward that should have been assigned to the provider is automatically redistributed to the auditor. All the above five steps are implemented as separate blockchain transactions on *pSAV* control plane.

III. *pSAV* Control Plane

This section describes how *pSAV* leverages blockchain as the trust foundation for inter-AS SAV services. Subscribers request SAV services from providers, while scattered auditors verify the effectiveness of SAV services. These processes, *i.e.*, registration, service subscription, and auditing, are recorded as transactions on the blockchain. Besides, a set of nodes acts as endorsers to authenticate the validity of transactions.

A. AS Registration: Verify Authenticity

All ASes must be registered before they join the blockchain. Various ASes can register as service subscribers, providers,

and auditors with public keys to identify themselves. The key of registration is to ensure the authenticity of the information provided by different ASes.

Service subscriber. Subscribers are often small ASes that do not have the technical ability to deploy a particular SAV technology or cannot do the job independently. Therefore, they can declare their packets' characteristics, *e.g.*, IP address, and request other ASes for SAV services by offering rewards. However, to avoid incorrectly filtering traffic belonging to other ASes, the declared IP address prefixes must be verified. To achieve this, we refer to [19] and implement blockchain-based IP address allocation and delegation to verify the IP prefixes declared by subscribers.

Service provider. Providers benefit from providing SAV services, *i.e.*, filtering spoofed traffic for other ASes. At registration, service providers need to declare the services they support and their prices. Besides, they are encouraged to provide their location or routing information for reference purposes. The services can be verified by auditors later, and providers offering substandard services will be penalized.

Service auditor. Each participating AS can be an auditor. Auditors gain benefits by verifying services provided by providers. By spoofing packets that should be discarded but get through to the provider, an auditor can prove that the SAV service is invalid. They can then submit the audit reports to the blockchain. After subscribers approve these reports, the rewards that should have been paid to the provider are redistributed to the auditor.

B. Service Subscription: Balance Auditability and Privacy

The service subscription process includes subscriber requests and provider response. Endorsers verify the validity and authenticity of the request and response transactions. First, the subscriber invokes a smart contract to pull provider information and select a service to subscribe to. Then, the subscriber constructs a subscription proposal that consists of the subscriber identity, protected IP address prefixes, the provider identity, the requested SAV service, the service start and end times, and other optional parameters. Upon receipt of the subscription proposal, endorsers verify the validity and authenticity of the transactions, *i.e.*, whether the proposal conforms to the endorsement rules. For example, the provider should support the subscribed service, and the IP addresses should belong to the subscriber. After a sufficient number of endorser signatures are collected, the legitimate proposal as a transaction is packaged by the orderer node and stored on the blockchain. After the subscription transaction is confirmed, the provider can accept or reject the provision of the requested service and invoke a smart contract to construct a response proposal. If accepting, providers need to update their packet processing rules on the data plane (§IV).

During the above service subscription process, the request and response transactions should be recorded in plain text on the blockchain to allow auditing of the service. However, some privacy information about specific filtering rules should not be disclosed in order to prevent attackers from using this information to steer by the SAV service. For example, legitimate packets carrying labels can be made public, while the method of label generation should not. For this purpose, we divide the proposal information into two parts. One part is plain text, while the other should be encrypted first with the public key of the other end. In this way, we can achieve both auditability and privacy.

C. Service Audit: Provide Incentives

All ASes can act as auditors and send spoofed packets, which providers should filter. Naturally, only auditors who give valid audit results will be rewarded. The key is who and how to confirm the audit result. First, the spoofed packets that should be filtered are either sent to the subscriber or indirectly cause response packets to be sent to the subscriber. Second, subscribers pay for SAV services and naturally expect the service to be trustworthy, i.e., they will not collide with auditors and trap providers. Therefore, the subscriber is the most suitable place to check audit results. Besides, to prevent other malicious auditors from plagiarizing the audit results, only the first approved auditor will be rewarded.

The auditing process consists of three steps. (1) Volunteer ASes pull the lists of subscribed SAV services from the blockchain. (2) They select some on-going services for auditing. The basic principle of the audit is to spoof packets that providers should filter, and the subscriber should know whether the provider filters the packets. Depending on the audited SAV service, the specific auditing method will be different. Auditors can communicate with the subscriber directly by spoofing the IP addresses of another AS. Theoretically, the communication will fail because providers should filter the spoofed packets. Notice that the path from the auditor to the subscriber should be through the provider to ensure the validity of spoofing. Auditors can also spoof the subscribers' IP addresses and send packets to other ASes. If the subscriber receives response packets, the SAV service is invalid. (3) After discovering the vulnerability of the SAV services, auditors can submit their reports to the blockchain. (4) The corresponding subscriber checks whether the audit is valid. Once the subscriber confirms that the audit result is valid, the smart contract will automatically redistribute the incentive INC that should have been paid to the provider. Subsequent audits by other auditors are no longer valid, since only the first acknowledged auditor receives rewards. Given an audit payoff ratio λ between 0 and 1, the auditor receives a portion of the incentive λINC, while the rest is returned to the subscriber. The rate defaults to 1 and can be predefined by the subscriber in the service subscription transaction.

IV. *pSAV* Data plane

This section introduces how *pSAV* implements various SAV services on the data plane using a P4-programmable switch as the border router. Figure 2 shows the placement of match-action tables. In the first stage, *PacketClassification Table* classifies packets with a type ID and decides which

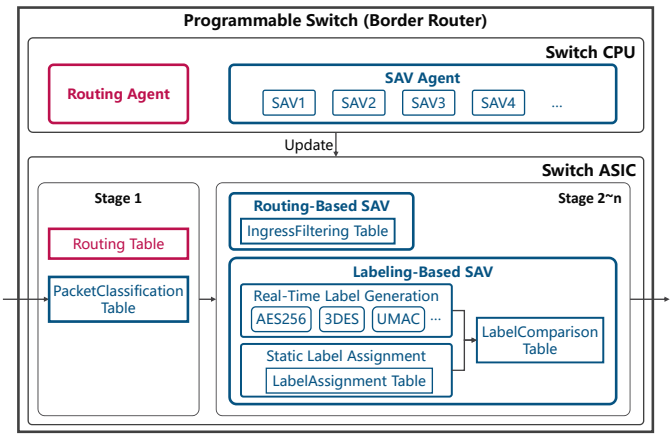


Fig. 2. **Match-action table placement on programmable switches to accommodate various SAV technologies.**

SAV service should be provided for received packets. For the former, each type ID is associated with some necessary control information for filtering, *e.g.*, the theoretical ingress port. For the latter, we define a separate 1-bit metadata field for each SAV service. If an SAV service is provided, the corresponding metadata should be set to 1. In general, the SAV service is AS-level, so the match field should include the IP prefix. Other packet fields can also be included when the service subscribers have more filtering requirements. Then the following stages implement specific SAV services, which will be explained later. On switch CPU, SAV agents interact with the blockchain clients in §III and update the ASIC in real time to provide or cancel specific SAV services. Existing SAV technologies can be generally classified into two methodologies, *i.e.*, routing-based and labeling-based. Next, we will detail how to implement them on programmable ASICs.

A. Routing-based SAV

The key idea of routing-based SAV is that while attackers may forge IP addresses, they have no control of the forwarding path of packets. Therefore, the border router can verify the authenticity according to the incoming direction of packets, *i.e.*, the ingress port. The legitimate routes decide the theoretical ingress port, either by learning directly from BGP updates [20] or by actively detecting the true inter-AS routes [21].

Traditionally, packet filtering and legitimate route learning are implemented in router hardware and software, respectively. The implementation is similar on programmable switches, as shown in Figure 2. To provide SAV services to various subscribers, *IngressFiltering Table* matches the type ID and ingress port exactly and decides whether to drop the packets. Considering multiple paths, each type ID may correspond to multiple theoretical ingress ports. Only packets hitting the table, *i.e.*, arriving from the correct port, are not dropped. The table entries are updated in real time by SAV agents on switch CPU. SAV agents obtain type ID-ingress port pairs directly from service subscribers or by performing specified learning algorithms. Since the implementation of SAV agents is similar to existing work, it is not discussed in this paper.

B. Labeling-based SAV

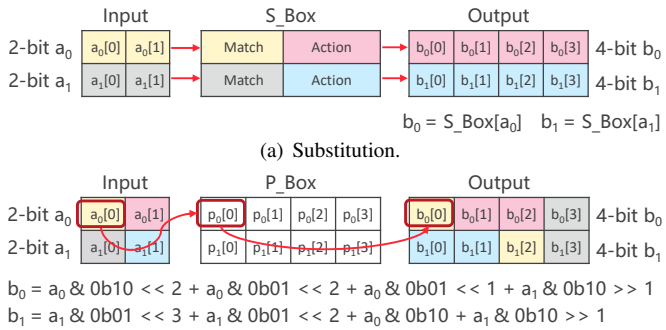
Labeling-based SAV verifies IP addresses based on mutually agreed tags. A legitimate packet with a real IP address should carry an exclusive label that can be verified. Labeling-based SAV implementation on programmable switches is quite intuitive, as shown in Figure 2. In general, we obtain the theoretical label that the packet should carry and then compare the calculated label with the actual label in the packet. If they are not equal, the packet should be discarded. Label generation methods are generally divided into two categories. The first category is static labeling. Packets with the same type ID should share the same label. For example, an AS can correspond to a single label, *i.e.*, all packets from/to the same AS carry the same label. Therefore, we can pre-store the labels of each type ID in *LabelAssignment Table*. To improve safety, the SAV agent may update the static labels periodically. The second type is dynamic labeling. Different packets have different labels. Labels are often generated by cryptographic computation and are related to packet fields, such as IP addresses, origin or destination AS, packet content. Since it is impossible to store a separate label for each packet, the label should be calculated in real time on the ASIC.

The challenge lies in *the gap between the limited programmability of programmable ASICs and the complexity of cryptography algorithms*. On the one hand, some common cryptography algorithms, *e.g.*, AES, are composed of many sequential operations with dependencies between them. On the other hand, programmable ASICs only support basic arithmetic operations, *e.g.*, *shift* and *xor*. In other words, many hardware stages are required to piece together those basic arithmetic operations to implement cryptography algorithms. However, the number of stages on the ASIC is limited. Although we can use *recirculation* to make a packet traverse the pipeline more than once, recirculation has the drawbacks of increasing packet delay and reducing bandwidth.

Given the above, next, we will describe how to support cryptography algorithms on the ASIC with as few pipeline stages as possible. Common encryption algorithms, *e.g.*, AES and 3DES, perform several transformation rounds to generate the final output, which is the label, based on some input fields. Each round consists of several processing steps. Although there are various types of symmetric encryption algorithms, they can be decomposed into four types of building blocks.

Substitution (S). The input block is divided into several multi-bit pieces. A substitution box (S-box) replaces the input piece with an output piece according to a non-linear transformation. The S-box can be implemented directly with an exact match table on programmable ASIC, as shown in Figure 3(a). Substitution takes up only one hardware stage, but the memory overhead incurred is relatively more substantial. Each input piece value corresponds to an exact match entry.

Permutation (P). The output units (*e.g.*, a bit) are rearranged based on input units according to a permutation box (P-box), as shown in Figure 3(b). A naive permutation implementation requires only one stage with many concurrent modification



(a) Substitution.

(b) Permutation.

Fig. 3. **Illustration of encryption steps.**

operations. However, considering permutation as a step of the whole encryption algorithm, one stage is insufficient. For example, if a bit is used as the basic unit of operation, we should first fetch each bit from the input pieces. For each output piece, first, we replace each bit value with the corresponding input bit. Then, we shift this bit according to the offset between this bit's position in the input and output piece. Finally, we sum all shifted values to get each output piece. The number of hardware stages required grows linearly with the width of the output piece and will increase with the number of transformation rounds. In contrast to this, we use exact match tables to implement permutation to reduce the stage overhead. A naive design uses one large exact match table that matches the input piece (a-bit) and gives the output piece (b-bit). Then we need 2^a table entries, each containing $a + b$ bits. To save memory, we can use more exact tables to match each part of the input piece separately and give the corresponding outputs. The final output should be the OR of all outputs. Assuming that each table matches c input bits, we need a/c tables. Each table contains 2^c entries, and each entry contains $c+b$ bits. The final result should be OR of c outputs, which requires $log_2 c$ stages.

MixColumn (M). Each input piece (column) is treated as a polynomial over a finite field and then multiplied with a fixed polynomial. Polynomial multiplication can be translated into entry multiplication and addition operations. Addition is XOR, while multiplication can be treated as a combination of XOR and shift. However, this implementation requires many hardware stages. According to matrix multiplication, each position in the output column is determined by only one input column. Therefore, we can use multiple exact match tables to implement MixColumn instead. Each input column corresponds to one exact match table, which modifies the value of affected output positions. Similar to the permutation operation, the final result should be the OR of all outputs.

KeyMixing (K). The input block is combined with a *subkey* using bitwise XOR. The subkeys are derived from a *main key* using a key schedule, which is an algorithm to generate subkeys. In *pSAV*, key schedule is implemented in each SAV agent on switch CPU and updates the subkeys in the form of table entries. Therefore, two stages are needed — one for subkey assignment and the other for XOR operations.

Common encryption algorithms such as DES, AES, and 3DES can be implemented by a combination of the above processing steps. Limited by the hardware stages in a pipeline, recirculation is needed to complete an encryption algorithm consisting of multiple encryption rounds.

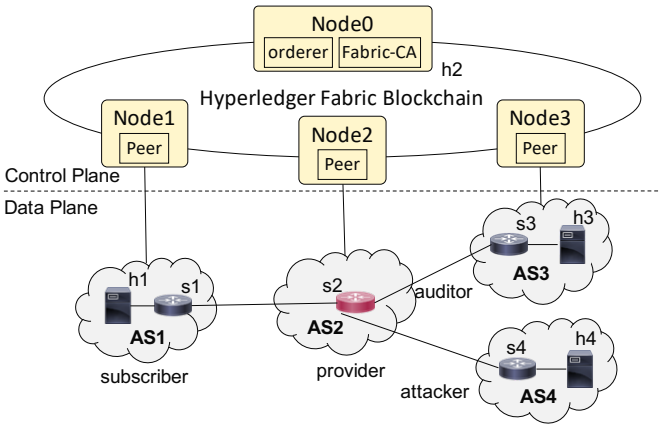
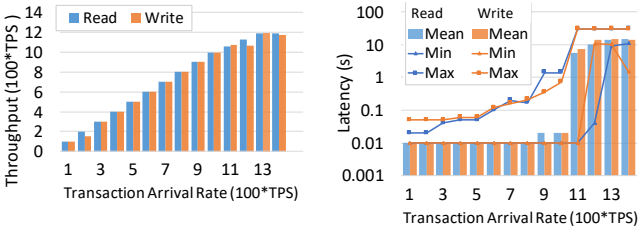


Fig. 4. **Evaluation test-bed.**



(a) Transaction throughput. (b) Transaction latency.

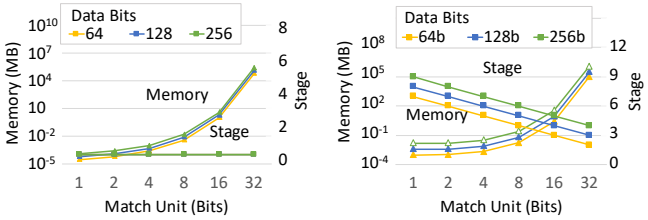
Fig. 5. **Transaction performance on the blockchain.**

V. Experimental Evaluation

We evaluate *pSAV* using 2 Barefoot Tofino switches and four servers, as shown in Figure 4. On the data plane, four ASes are the subscriber, provider, auditor, and attacker, respectively. AS2 acts as a service provider and provides SAV services on its border router s2, which is a Tofino switch. We emulate three border routers (s1, s3, s4) with one Tofino switch by allocating individual ports to each emulated router. Further details about this switch emulation can be found in [22]. All links are 10 Gbps. On the control plane, to facilitate the access control for participating ASes, we use Hyperledger Fabric to build our test blockchain. We implement SAV contracts with 650 lines of GO language to simulate service subscription and audit. We run four virtual machines within a single physical server and run a *pSAV* blockchain node in each of them. All virtual machine nodes are running on Ubuntu 18.04. One node acts as the orderer and offers Fabric CA functionality, and the other three nodes correspond to three participant ASes (AS1, AS2, AS3).

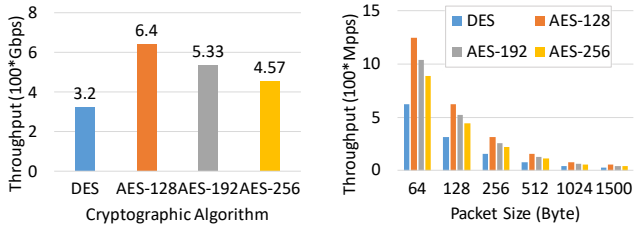
A. Blockchain Performance on Control Plane

In this part, we test the blockchain performance, including the transaction throughput and latency on the control plane. All transactions can be generally classified into two types, *i.e.*, writes and reads. Read operations differ from write operations because the blockchain state does not change. Write-related



(a) Substitution. (b) Permutation and MixColumn.

Fig. 6. **Memory and stage overhead of various operations.**



(a) Gbps under all packet sizes. (b) Mpps under different packet sizes.

Fig. 7. **Maximum throughput of labeling-based SAV with various cryptographic algorithms.**

transactions include registration, service request from the subscriber, response from the provider, audit result submission from the auditor, and confirmation from the subscriber. Below we compare the performance of read and write operations.

Figure 5(a) shows the transaction throughput for different transaction sending rates. We can see that the read throughput is slightly larger than write throughput. When the transaction sending rate is small, both the read throughput and write throughput increase with the sending rate, and most transactions can be finished in time. However, when the sending rate exceeds 1100 transactions per second (TPS), the throughput no longer increases. Figure 5(a) shows the minimum, maximum, and mean transaction latency for different transaction sending rates. We can see that when the transaction is not frequent, *e.g.*, smaller than 900 TPS, both the read and write latency is smaller than one second. The latency increases sharply when the transaction rate exceeds 900 TPS and can be tens of seconds. Based on the above, we can conclude that currently, *pSAV* can easily support hundreds of transactions per second with no more than one-second latency.

B. SAV Performance and Overhead on Data Plane

In this part, we evaluate the performance and overhead of SAV implementation on the data plane. We mainly consider the labeling-based SAV implementation, which relies on cryptographic computation, since other situations, such as static labels and routing-based SAV, are quite simple.

1) Memory and Stage Overhead: Recall that we break down cryptographic algorithms into different blocks and use exact match tables to reduce the stage overhead in the cost of increased memory usage IV-B. Figure 6 shows the memory and stage overhead of each block by assuming that the input and output data are of the same width, *i.e.*, 64, 128, and 256 bits. We can see that for permutation and mixcolumn operations, with the match unit width increasing, they require more SRAM and fewer stages. However, for substitution, the stage number is always one. When we take an 8-bit match unit, even 128-bit permutation and mixcolumn operation require five stages and less than 0.1 MB SRAM, which is far smaller than the current available SRAM in programmable switches.

2) Throughput: We implement the label generation with various cryptography algorithms on the Tofino switch. Based on the total stage number in one pipeline and the required stage number for label generation, we can obtain the recirculation number. Recall that the Tofino switch has two reserved 100 Gbps loopback ports, while all other physical ports can also be configured as loopback mode. Therefore, the theoretical maximum throughput is equal to the total port bandwidth (3.4 Tbps for a 2-pipe switch or 6.6 Tbps for a 4-pipe switch) divided by the required recirculation number. Figure 7(a) shows the maximum throughput (Gbps) using cryptographic algorithms including DES (9 recirculations), AES-128 (4 recirculations), AES-192 (5 recirculations), AES-256 (6 recirculations). We can see that DES achieves smaller throughput (320Gbps) than AES since DES has the most encryption rounds (16). The throughput of AES decreases as the encryption round increases, and AES-256 has the smallest throughput (457 Gbps). By comparison, the software cryptography implementation by DPDK achieves no more $<$ 30 Gbps throughput, and the Intel Cryptodev QAT (Intel QuickAssist Technology) achieves $<$ 100 Gbps throughput with hardware acceleration [23]. Figure 7(a) further considers packet sizes and shows the Mpps (M packet per second) results considering that programmable switches can process various packet sizes at line rate. From the above, we can conclude that the programmable switch can achieve larger throughput than existing software (DPDK SW) and hardware (Intel QAT) cryptographic implementation, benefiting from the large total bandwidth of programmable switches. Larger throughput means larger service capacity and also more revenues from the perspective of service providers.

VI. Related Work

Inter-AS SAV technologies. Routing-based SAV technologies [20], [24] verify the authenticity of source IP addresses according to the incoming direction of packets, *i.e.*, the ingress port. Labeling-based SAV technologies [9], [25]–[27] assume that legitimate packets with authentic source IP addresses should carry an exclusive cryptographic label and verifies source IP addresses based on cryptography. This work focus on specific SAV technologies. apf [15] is closest to our work. We both take inter-AS SAV as a service to promote deployment. Compared with apf, we solve the trust foundation issue fundamentally by leveraging blockchain to provide automotive incentive allocation. Besides, we further promote deployment by introducing the programmable switches to implement SAV techniques.

P4-programmable switch. Programmable switches offer excellent performance such as high throughput and low latency [16], [17], and sufficient programmability via allowing users to flexibly define the operation for each packet, including the parser, the pipeline, and the deparser. Such flexibility

has motivated much research work to enhance network functions and applications with programmable switches. [28]–[30] offload network functions such as load balancing, heavy-hitter detection, and monitoring to programmable ASICs, while [31]–[34] explore accelerating Internet applications and securing networks.

Blockchain-based auditing. The distributed trust model motivates much research work to utilize blockchain to implement auditing schemes. [35] proposes a blockchain-based incentivizing and auditing framework for decentralized storage. [36] proposes a deduplicatable data auditing mechanism using blockchain. [37] proposes a blockchain-based public and efficient certificate audit scheme for TLS connections, [38], [39] utilizes blockchain to audit released IoT systems. In this paper, we build a blockchain-based SAV platform to provide decentralized and auditable SAV services with automotive incentives,

VII. Conclusion

In this paper, we propose *pSAV*, a practical and decentralized inter-AS SAV service framework to promote inter-AS SAV deployment. *pSAV* takes SAV as a payable service and divides participant ASes into service subscribers, providers, and auditors. Leveraging blockchain as the trust foundation, *pSAV* enables service providers accountable for their offered services. Meanwhile, *pSAV* allocates incentives to auditors for detecting unqualified services. Besides, *pSAV* proposes a flexible, high-performance, and low-cost SAV service implementation mechanism for providers.

References

[1] Toby Ehrenkranz and Jun Li. On the state of ip spoofing defense. *ACM Transactions on Internet Technology (TOIT)*, 9(2):1–29, 2009.

[2] Sam Kottler. February 28th ddos incident report. *GitHub Engineering*, 2018.

[3] Ben Stock, Giancarlo Pellegrino, et al. Hey, you have a problem: On the feasibility of large-scale web vulnerability notification. In *Proceedings of USENIX Security 2016*, pages 1015–1032, 2016.

[4] Jianping Wu, Jun Bi, et al. A source address validation architecture (sava) testbed and deployment experience. *RFC5210*, 2008.

[5] Jianping Wu, J Bi, et al. Source address validation improvement (savi) framework. *RFC7039*, 2013.

[6] P Ferguson. Defeating denial of service attacks which employ ip source address spoofing. *RFC 2827*, 2000.

[7] Fred Baker and Pekka Savola. Ingress filtering for multihomed networks. Technical report, BCP 84, RFC 3704, March, 2004.

[8] Robert Beverly, Arthur Berger, et al. Understanding the efficacy of deployed internet source address validation filtering. In *Proceedings of the 9th ACM SIGCOMM conference on Internet measurement*, pages 356–369, 2009.

[9] Anat Bremler-Barr and Hanoch Levy. Spoofing prevention method. In *Proceedings IEEE 24th Annual Joint Conference of the IEEE Computer and Communications Societies.*, volume 1, pages 536–547. IEEE, 2005.

[10] Xin Liu Ang Li Xiaowei Yang and David Wetherall. Passport: Secure and adoptable source authentication, 2008.

[11] Kihong Park and Heejo Lee. On the effectiveness of route-based packet filtering for distributed dos attack prevention in power-law internets. *ACM SIGCOMM computer communication review*, 31(4):15–26, 2001.

[12] Michalis Faloutsos, Petros Faloutsos, and Christos Faloutsos. On power-law relationships of the internet topology. *ACM SIGCOMM computer communication review*, 29(4):251–262, 1999.

[13] Zhenhai Duan, Xin Yuan, et al. Controlling ip spoofing through interdomain packet filters. *IEEE transactions on Dependable and Secure computing*, 5(1):22–36, 2008.

[14] Matthew Luckie, Robert Beverly, et al. Network hygiene, incentives, and regulation: deployment of source address validation in the internet. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 465–480, 2019.

[15] Y. Jia, Y. Liu, and G. Ren. Address protection-as-a-service an inter-as framework for ip spoofing resilience. In *2019 IEEE Global Communications Conference (GLOBECOM)*, pages 1–7, 2019.

[16] The P4 Language Consortium. The p4 language specificatio. Website. https://goo.gl/Sq3TRK.

[17] Barefoot Networks. Tofino. Website, 2019. https://www.barefootnetworks.com/products/brief-tofino/.

[18] Elli Androulaki, Artem Barger, et al. Hyperledger fabric: a distributed operating system for permissioned blockchains. In *Proceedings of the thirteenth EuroSys conference*, pages 1–15, 2018.

[19] Jordi Paillisse, Miquel Ferriol, et al. Ipchain: Securing ip prefix allocation and delegation with blockchain. In *2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, pages 1236–1243. IEEE, 2018.

[20] Zhenhai Duan, Xin Yuan, et al. Constructing inter-domain packet filters to control ip spoofing based on bgp updates. In *INFOCOM*, 2006.

[21] Jun Li, Jelena Mirkovic, et al. Learning the valid incoming direction of ip packets. *Computer Networks*, 52(2):399–417, 2008.

[22] Jiamin Cao, Yu Zhou, et al. Turbonet: Faithfully emulating networks with programmable switches. In *2020 IEEE 28th International Conference on Network Protocols (ICNP)*, pages 1–11. IEEE, 2020.

[23] Dpdk intel cryptodev performance report.

[24] Paul Ferguson and Daniel Senie. rfc2827: network ingress filtering: defeating denial of service attacks which employ ip source address spoofing, 2000.

[25] Stephen Kent and Randall Atkinson. Ip authentication header, 1998.

[26] Lin He, Gang Ren, Ying Liu, and Jiahai Yang. Pavi: Bootstrapping accountability and privacy to ipv6 internet. *IEEE/ACM Transactions on Networking*, 29(2):695–708, 2021.

[27] Yihao Jia, Ying Liu, Gang Ren, and Lin He. Revisiting inter-as ip spoofing let the protection drive source address validation. In *2017 IEEE 36th International Performance Computing and Communications Conference (IPCCC)*, pages 1–10, 2017.

[28] Naga Katta, Mukesh Hira, et al. Hula: Scalable load balancing using programmable data planes. In *Proceedings of SOSR*. ACM, 2016.

[29] Theo Jepsen, Masoud Moshref, et al. Packet subscriptions for programmable asics. In *Proceedings of HotNets*. ACM, 2018.

[30] Yu Zhou, Jun Bi, Tong Yang, Kai Gao, and Cheng Zhang. Keysight: Troubleshooting programmable switches via scalable high-coverage behavior tracking. In *Proceedings of ICNP*. IEEE, 2018.

[31] Xin Jin, Xiaozhou Li, et al. Netcache: Balancing key-value stores with fast in-network caching. In *Proceedings of SOSR*. ACM, 2017.

[32] Xin Jin, Xiaozhou Li, et al. Netchain: Scale-free sub-rtt coordination. In *Proceedings of NSDI*. USENIX, 2018.

[33] Yu Zhou, Zhaowei Xi, et al. Hypertester: High-performance network testing driven by programmable switches. In *Proceedings of CoNEXT*. ACM, 2019.

[34] Peng Kuang, Ying Liu, and Lin He. P4dad: Securing duplicate address detection using p4. In *Proceedings of IEEE ICC*, 2020.

[35] Yuefeng Du, Huayi Duan, Anxin Zhou, Cong Wang, Man Ho Au, and Qian Wang. Towards privacy-assured and lightweight on-chain auditing of decentralized storage. *arXiv preprint arXiv:2005.05531*, 2020.

[36] Yang Xu, Cheng Zhang, Guojun Wang, Zheng Qin, and Quanrun Zeng. A blockchain-enabled deduplicatable data auditing mechanism for network storage services. *IEEE Transactions on Emerging Topics in Computing*, pages 1–1, 2020.

[37] Jing Chen, Shixiong Yao, Quan Yuan, Kun He, Shouling Ji, and Ruiying Du. Certchain: Public and efficient certificate audit based on blockchain for tls connections. In *Proceedings of IEEE INFOCOM*, 2018.

[38] Bo Wu, Ke Xu, et al. Smartcrowd: Decentralized and automated incentives for distributed iot system detection. In *39th IEEE International Conference on Distributed Computing Systems, ICDCS 2019, Dallas, TX, USA, July 7-10, 2019*, pages 1106–1116. IEEE, 2019.

[39] Bo Wu, Qi Li, et al. Smartretro: Blockchain-based incentives for distributed iot retrospective detection. In *15th IEEE International Conference on Mobile Ad Hoc and Sensor Systems, MASS 2018, Chengdu, China, October 9-12, 2018*, pages 308–316. IEEE Computer Society, 2018.